

1 DE ENERO DE 2023



INFORME DE PRÁCTICAS MA

Mecánica aplicada

Mario Sánchez Rodríguez y Daniel Gómez Parreño

Índice

Estudio de Cinemática en Matlab:

1. Esquema del mecanismo identificando cada pieza con números y cada junta con letras. Es este esquema se deben indicar las dimensiones más importantes del mecanismo.
2. Cálculo de los grados de libertad del mecanismo.
3. Esquemas de las piezas en los que se localice el sistema de coordenadas local, se identifiquen las variables del sistema y se indiquen claramente las dimensiones de las barras y placas (las dimensiones necesarias para la formulación de las ecuaciones de restricción).
4. Ecuaciones de restricción de cada junta y ecuación motriz de la cinemática.
5. Matriz jacobiana.
6. Vector de posición inicial, resultado de posición tras la primera aplicación del algoritmo Newton-Raphson indicando número de iteraciones realizadas y error permitido.
7. Vector VEL (para la resolución de la velocidad) y vector gamma (para la resolución de la aceleración).

Dinámica en Matlab:

8. Descripción de las fuerzas y momentos aplicados sobre el mecanismo.
9. Masas e inercias de cada pieza.

Modelos en Inventor:

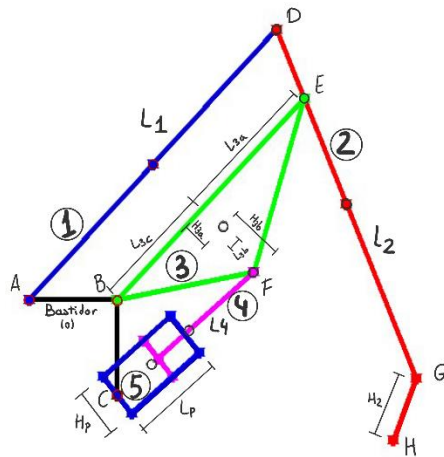
10. Esquemáticamente indicar las relaciones entre las piezas en entorno ensamblaje (restricciones) y en el entorno simulación (uniones).
11. Indicar el material utilizado en cada pieza.
12. Indicar como se han aplicado el movimiento impuesto en la cinemática y las cargas externas en la dinámica.

Mostrar y analizar los resultados:

13. Mostrar los resultados de las simulaciones **cinemáticas** realizadas con Matlab y con Inventor: posición, velocidad y aceleración de las variables del sistema. Se deben plotear los resultados y realizar un análisis crítico de ellos.
14. Mostrar los resultados de las simulaciones **dinámicas** realizadas con Matlab y con Inventor: posición, velocidad y aceleración de las variables del sistema. Se deben plotear los resultados y realizar un análisis crítico de ellos. En los dos puntos anteriores incluir el análisis comparativo entre los dos métodos de simulación.
15. Códigos de Matlab.

Estudio de Cinemática en Matlab:

1. Esquema del mecanismo identificando cada pieza con números y cada junta con letras. En este esquema se deben indicar las dimensiones más importantes del mecanismo.



Longitud en metros (m)

L1=4;	L2=4;
L3a=1.6;	H3a=0.3;
L3b=0.192;	H3b=0.589;
L3c=1.4;	L4=1.9161;
L5=1;	H2=0.3;
Lp=1;	Hp=0.5;
A(0,0); B(1,0); C(1,-1);	

2. Cálculo de los grados de libertad del mecanismo.

$$GDL = 3m * -2P1 - P2$$

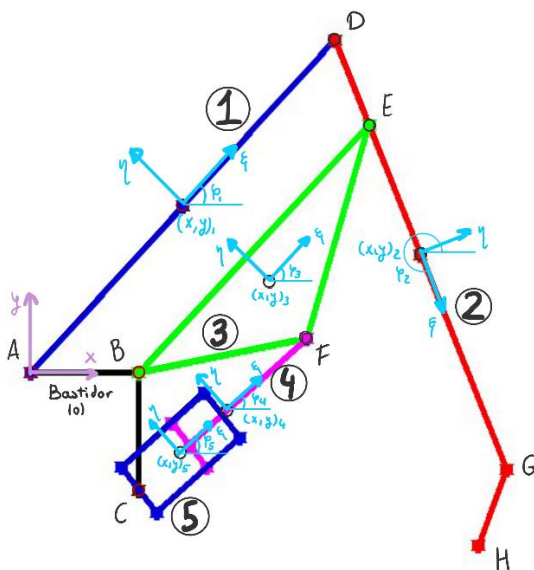
m es la cantidad de barras del mecanismo exceptuando el bastidor. m=7

P1 es la cantidad de pares de rotación más la de traslación. P1=10

P2 es el número de pares cinemáticos de traslación rotación. P2=0

$$GDL = 1$$

3. Esquemas de las piezas en los que se localice el sistema de coordenadas local, se identifiquen las variables del sistema y se indiquen claramente las dimensiones de las barras y placas (las dimensiones necesarias para la formulación de las ecuaciones de restricción).



4. Ecuaciones de restricción de cada junta y ecuación motriz de la cinemática.

Estas variables hacen referencia a los dibujos aportados con anterioridad.

A continuación, definimos el vector:

$$q() = \begin{bmatrix} x_1 \\ y_1 \\ \varphi_1 \\ x_2 \\ y_2 \\ \varphi_2 \\ x_3 \\ y_3 \\ \varphi_3 \\ x_4 \\ y_4 \\ \varphi_4 \\ x_5 \\ y_5 \\ \varphi_5 \end{bmatrix}$$

Y por último se presentan las 15 ecuaciones de restricción que rigen el mecanismo, incluyendo la motriz.

[illegible]

5. Matriz jacobiana.

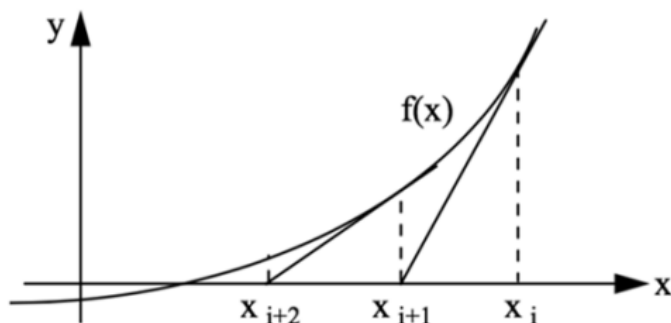
$$\Phi_q = \begin{bmatrix} 1 & 0 & L1/2 * \sin(q(3)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -L1/2 * \cos(q(3)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & -L1/2 * \sin(q(3)) & -1 & 0 & -L2/2 * \sin(q(6)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & L1/2 * \cos(q(3)) & 0 & -1 & L2/2 * \cos(q(6)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0.3015 * L2 * \sin(q(6)) & -1 & 0 & L3a * \sin(q(9)) + H3a * \cos(q(9)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -0.3015 * L2 * \cos(q(6)) & 0 & -1 & -L3a * \cos(q(9)) + H3a * \sin(q(9)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & L3c * \sin(q(9)) - H3a * \cos(q(9)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -L3c * \cos(q(9)) - H3a * \sin(q(9)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & L3b * \sin(q(9)) + H3b * \cos(q(9)) & -1 & 0 & L4/2 * \sin(q(12)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -L3b * \cos(q(9)) + H3b * \sin(q(9)) & 0 & -1 & -L4/2 * \cos(q(12)) & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -q(14) - 1 & q(13) - 1 & 0 & q(11) + 1 - q(10) + 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & L5/2 * \sin(q(15)) & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -L5/2 * \cos(q(15)) & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

6. Vector de posición inicial, resultado de posición tras la primera aplicación del algoritmo Newton-Raphson indicando número de iteraciones realizadas y error permitido.

El vector de posición inicial ha sido medido en un dibujo a escala del mecanismo para extraer las coordenadas de cada uno de los centros de masa de las barras y placas. Se define por:

$$q_0 = \begin{bmatrix} 1.414 \\ 1.414 \\ \frac{\pi}{4} \\ 3.62 \\ 0.992 \\ \pi \cdot \frac{293}{180} \\ 1.787982 \\ 1.021084 \\ \frac{\pi}{4} \\ 1.93666 \\ -0.23666 \\ \pi \cdot \frac{39}{180} \\ 1.3875 \\ -0.68406 \\ \pi \cdot \frac{39}{180} \end{bmatrix} \quad \text{Tras 3 Iteraciones: } q_0 = \begin{bmatrix} 1.4142 \\ 1.4142 \\ 0.7854 \\ 3.6208 \\ 0.9921 \\ 5.1198 \\ 2.2100 \\ 0.7654 \\ 0.7751 \\ 1.7424 \\ -0.3949 \\ 0.6838 \\ 1.3876 \\ -0.6841 \\ 0.6838 \end{bmatrix}$$

Posteriormente se procede siguiendo el método iterativo de Newton-Raphson. El proceso iterativo parte del q_0 (aprox) y a medida que se resuelve la ecuación nos aproximamos más a la q que recoge la posición inicial del sistema.



- Definimos el vector VEL como:
- $$VEL = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ w \end{bmatrix}, \text{ siendo } w = 100 \text{ rad/s.}$$

$$\gamma = \begin{bmatrix} (L1/2 * \cos(q(3))) * v(3)^{\wedge}2 \\ (L1/2 * \sin(q(3))) * v(3)^{\wedge}2 \\ -(L1/2 * \cos(q(3))) * v(3)^{\wedge}2 - (L2/2 * \cos(q(6))) * v(6)^{\wedge}2 \\ -(L1/2 * \sin(q(3))) * v(3)^{\wedge}2 - (L2/2 * \sin(q(6))) * v(6)^{\wedge}2 \\ (0.3015 * L2 * \cos(q(6))) * v(6)^{\wedge}2 + (L3a * \cos(q(9)) - H3a * \sin(q(9))) * v(9)^{\wedge}2 \\ (0.3015 * L2 * \sin(q(6))) * v(6)^{\wedge}2 + (L3a * \sin(q(9)) - H3a * \cos(q(9))) * v(9)^{\wedge}2 \\ (L3c * \cos(q(9)) + H3a * \sin(q(9))) * v(9)^{\wedge}2 \\ (L3c * \sin(q(9)) - H3a * \cos(q(9))) * v(9)^{\wedge}2 \\ (L3b * \cos(q(9)) - H3b * \sin(q(9))) * v(9)^{\wedge}2 + (L4/2 * \cos(q(12))) * v(12)^{\wedge}2 \\ (L3b * \sin(q(9)) + H3b * \cos(q(9))) * v(9)^{\wedge}2 + (L4/2 * \sin(q(12))) * v(12)^{\wedge}2 \\ 0 \\ -(v(14)) * v(10)^{\wedge}2 + (v(13)) * v(11)^{\wedge}2 + (v(11)) * v(13)^{\wedge}2 + (-v(10)) * v(14)^{\wedge}2 \\ (L5/2 * \cos(q(15))) * v(15)^{\wedge}2 \\ (L5/2 * \sin(q(15))) * v(15)^{\wedge}2 \\ 0 \end{bmatrix}$$

Velocidad

$$\frac{d\Phi}{dt} = \Phi_q \cdot \dot{q} = 0 \quad \begin{bmatrix} \Phi_q \\ \Phi_q^d \end{bmatrix} \cdot \dot{q} = \begin{bmatrix} 0 \\ -\Phi_t^d \end{bmatrix}$$

$$\frac{d^2\Phi}{dt} = (\Phi_q \cdot \dot{q}) \cdot \dot{q} + \Phi_q \cdot \ddot{q} = 0 \quad \begin{bmatrix} \Phi_q \\ \Phi_q^d \end{bmatrix} \cdot \ddot{q} = \begin{bmatrix} \gamma \\ -(\Phi_q^d \cdot \dot{q}) - 2 \cdot \Phi_{qt}^d \cdot \dot{q} - \Phi_{tt}^d \end{bmatrix}$$

Dinámica en Matlab:

8. Descripción de las fuerzas y momentos aplicados sobre el mecanismo.

$$G_{ext} = \begin{bmatrix} Fx \\ Fy \\ M \\ Fx \\ Fy \\ M \\ Fx \\ Fy \\ M \\ Fx \\ Fy \\ M \end{bmatrix} = \begin{bmatrix} 0 \\ -m1 * g \\ M1 \\ 0 \\ -m2 * g \\ 0 \\ 0 \\ -m3 * g \\ 0 \\ 0 \\ -m4 * g \\ 0 \\ 0 \\ -m5 * g \\ 0 \end{bmatrix}$$

donde $g=9,8 \text{ m}\cdot\text{s}^{-2}$, el valor de cada masa está indicado en el siguiente apartado y el momento aplicado en la barra 1 (M1) posee un valor de $10 \text{ N}\cdot\text{m}$.

9. Masas e inercias de cada pieza.

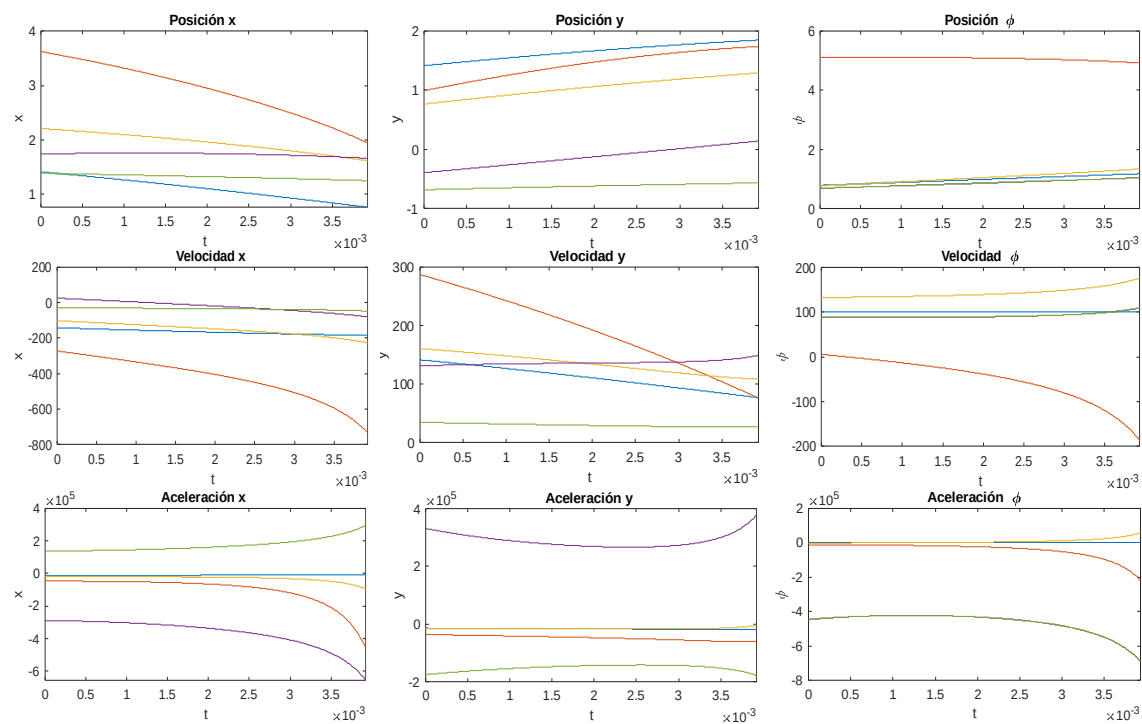
	Bastidor	Barra 1	Barra 2	Placa 3	Barra 4	Deslizadera 5
Masa (Kg)	388,260	176,26	216,405	282,446	55,036	54,872
Inercia(m^4)	0	259,791	378,711	203,679	27,17	26,598

Mostrar y analizar los resultados:

13. Mostrar los resultados de las simulaciones cinemáticas realizadas con Matlab y con inventor: posición, velocidad y aceleración de las variables del sistema. Se deben plotear los resultados y realizar un análisis crítico de ellos. (incluir el análisis comparativo entre los dos métodos de simulación)

MATLAB

	x	y	fi
Pos	Tabla pos (x)	Tabla pos (y)	Tabla pos (fi)
Vel	Tabla vel (x)	Tabla vel (y)	Tabla vel (fi)
acel	Tabla acel (x)	Tabla acel (y)	Tabla acel (fi)

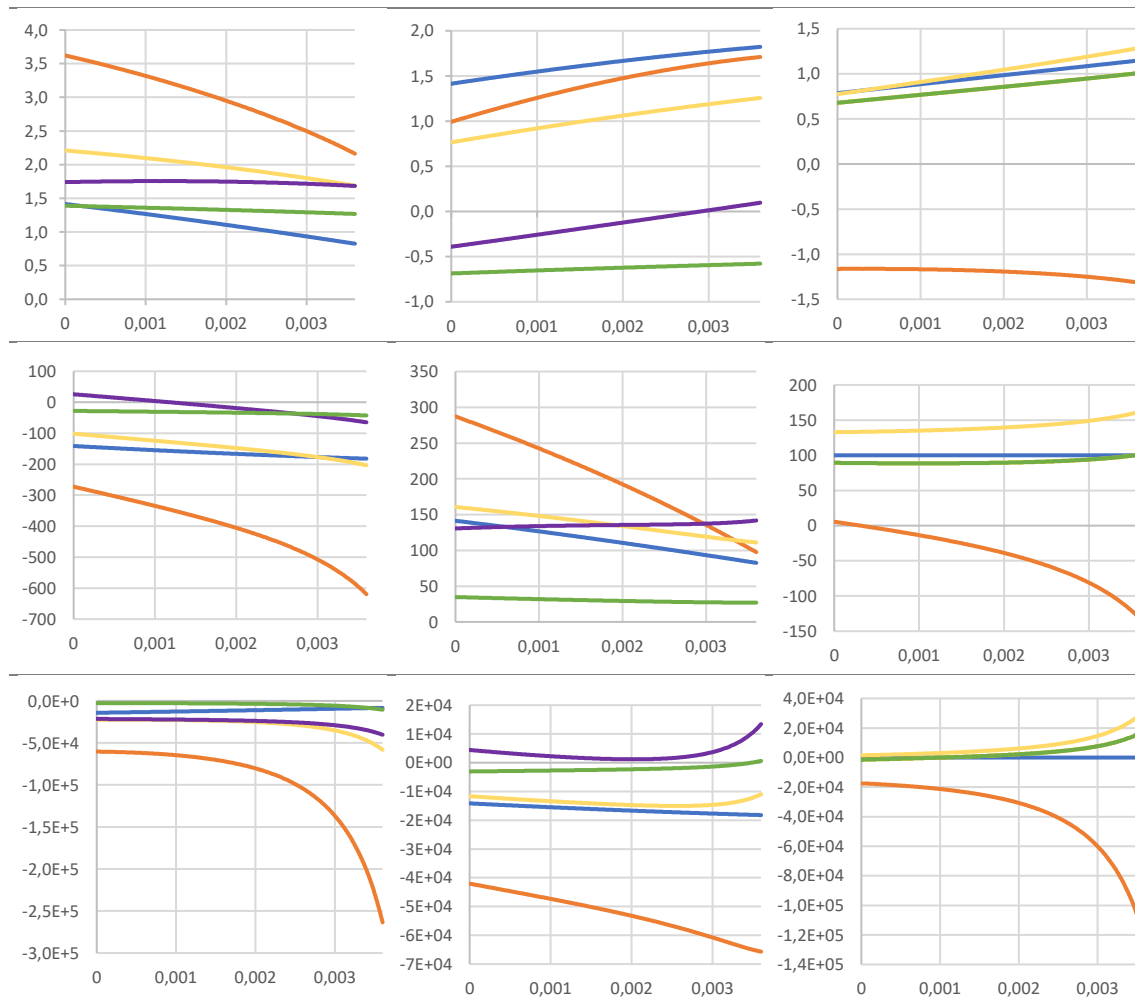


INVENTOR

Siguiendo el esquema previamente presentado:

Se utiliza la siguiente disposición en cada una de las gráficas obtenidas, asignando un color a cada una de las barras para mejor comprensión.

— barra 1
— barra 2
— barra 3
— barra 4
— barra 5



Como se puede apreciar, posición y velocidad son exactamente iguales en Matlab e Inventor, por lo que la simulación ha sido realizada con éxito.

Sin embargo, las aceleraciones no coinciden. Esto puede deberse a un fallo en el gamma o las ecuaciones para hallar la aceleración en Matlab o incluso en Inventor, error que no hemos sido capaces de solucionar. Este mismo error también se encuentra en dinámica.

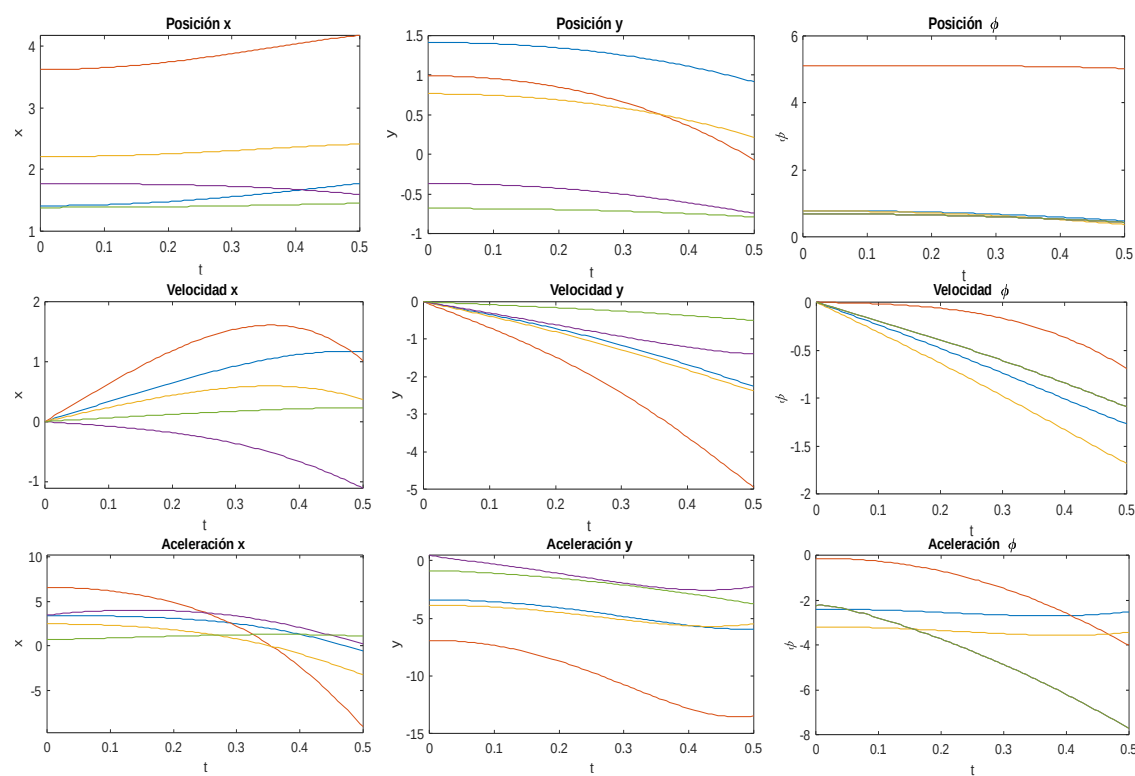
Estas gráficas simplemente representan la posición velocidad y aceleración de las componentes cartesianas x e y, así como el ángulo ϕ en que se encuentran los centros de masas de cada una de las barras del mecanismo a estudiar.

En la cinemática, se estudia un movimiento a velocidad constante y controlada de la barra motora, que en nuestro caso es la barra 1.

14. Mostrar los resultados de las simulaciones dinámicas realizadas con Matlab y con Inventor: posición, velocidad y aceleración de las variables del sistema. Se deben plotear los resultados y realizar un análisis crítico de ellos. (incluir el análisis comparativo entre los dos métodos de simulación)

MATLAB

	x	y	fi
Pos	Tabla pos (x)	Tabla pos (y)	Tabla pos (fi)
Vel	Tabla vel (x)	Tabla vel (y)	Tabla vel (fi)
acel	Tabla acel (x)	Tabla acel (y)	Tabla acel (fi)

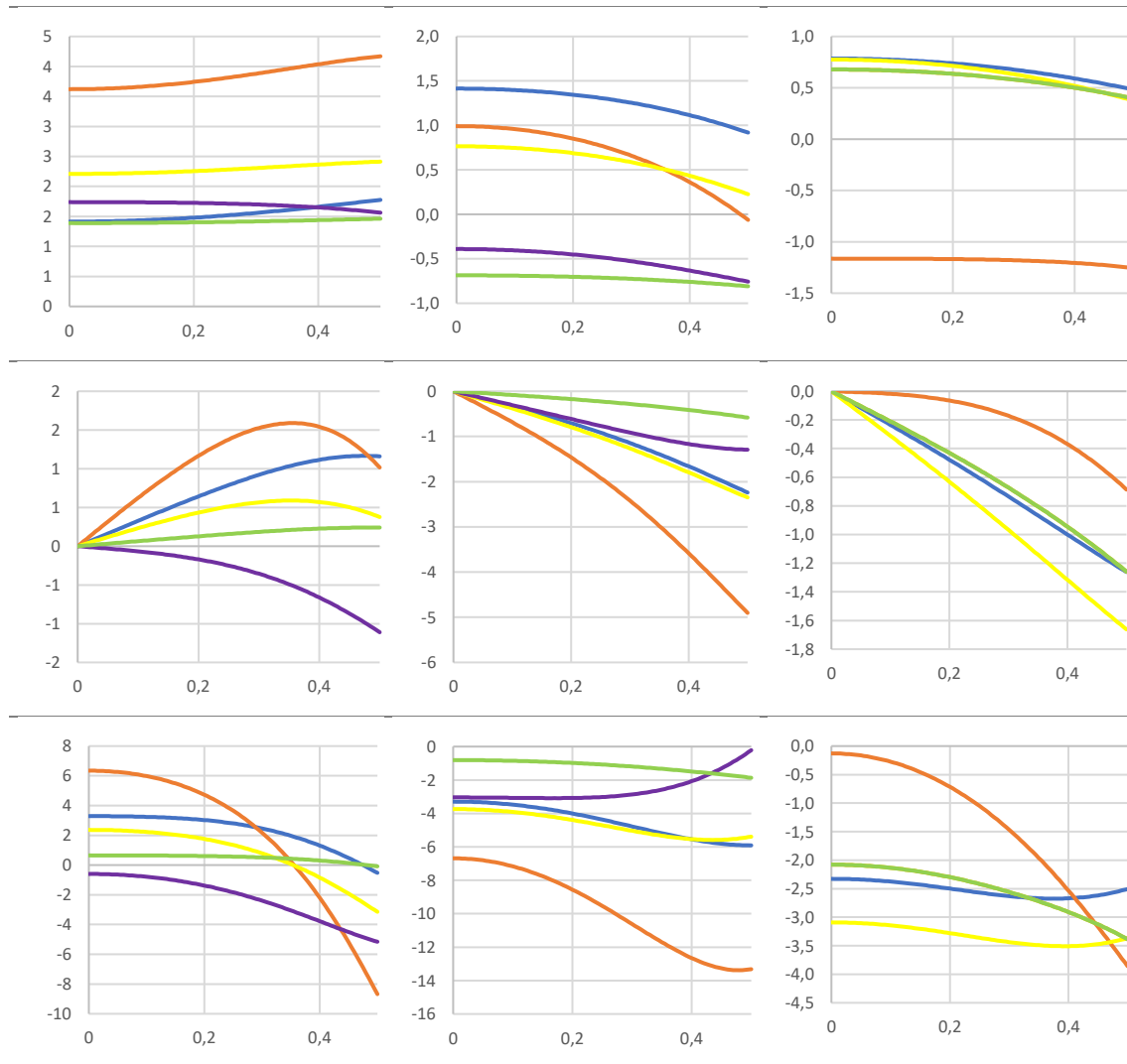


INVENTOR

Siguiendo el esquema previamente presentado:

Se utiliza la siguiente disposición en cada una de las gráficas obtenidas, asignando un color a cada una de las barras para mejor comprensión.

— barra 1
— barra 2
— barra 3
— barra 4
— barra 5



Estas gráficas representan la posición velocidad y aceleración de las componentes cartesianas x e y, así como el ángulo ϕ en que se encuentran los centros de masas de cada una de las barras del mecanismo a estudiar.

En la dinámica se estudia el movimiento del mecanismo accionado por su propio peso y, en nuestro caso, un momento externo aplicado en sentido horario a la unión de la barra 1 con el bastidor.

Ambas gráficas tienen sentido; tanto la de Matlab como la de Inventor, pues el movimiento es muy similar. A pesar de ello, las aceleraciones no coinciden y suponemos que se debe al mismo fallo que provoca este mismo efecto en el análisis cinemático.

15. Códigos de Matlab.

·RESTRICCIONES

```
function F=restricciones(q)
% Función restricciones. Tarea Mecanica Aplicada
% Prácticas 1 y 2

global L1 L2 L3a H3a L3b H3b L3c L4 L5 H2 w t i

% L1=4;
% L2=4;
% L3a=1.6;
% L3c=1.4;
% H3a=0.3;
% L3b=0.192;
% H3b=0.589;
% L4=1.9161;
% L5=1;
% H2=0.3;

F=[q(1)-L1/2*cos(q(3));
   q(2)-L1/2*sin(q(3)); %2

   q(1)+L1/2*cos(q(3))-q(4)+L2/2*cos(q(6));
   q(2)+L1/2*sin(q(3))-q(5)+L2/2*sin(q(6)); %4

   q(4)-0.3015*L2*cos(q(6))-q(7)-L3a*cos(q(9))+H3a*sin(q(9));
   q(5)-0.3015*L2*sin(q(6))-q(8)-L3a*sin(q(9))-H3a*cos(q(9)); %6

   q(7)-L3c*cos(q(9))-H3a*sin(q(9))-1;
   q(8)-L3c*sin(q(9))+H3a*cos(q(9)); %8

   q(7)-L3b*cos(q(9))+H3b*sin(q(9))-q(10)-L4/2*cos(q(12));
   q(8)-L3b*sin(q(9))-H3b*cos(q(9))-q(11)-L4/2*sin(q(12)); %10

   q(12)-q(15);
   q(13)*q(11)-q(13)*q(14)-q(11)+q(14)-q(10)*q(14)-q(10)+q(13)*q(14)+q(13);
%12

   q(13)-L5/2*cos(q(15))-1;
   q(14)-L5/2*sin(q(15))+1; %14
   q(3)-pi/4-w*t(i)]; %anular -w*t(i) solamente para posición_trabajo.m

return
```

·JACOBIANA

```
function J=jacobiana(q)
% Función jacobiana. Tarea Mecanica Aplicada
% Prácticas 1 y 2

global L1 L2 L3a H3a L3b H3b L3c L4 L5 H2

% L1=4;
% L2=4;
```

```

% L3a=1.6;
% L3c=1.4;
% H3a=0.3;
% L3b=0.192;
% H3b=0.589;
% L4=1.9161;
% L5=1;
% H2=0.3;

J=[1 0 L1/2*sin(q(3)) 0 0 0 0 0 0 0 0 0 0 0;
   0 1 -L1/2*cos(q(3)) 0 0 0 0 0 0 0 0 0 0 0;    %2

   1 0 -L1/2*sin(q(3)) -1 0 -L2/2*sin(q(6)) 0 0 0 0 0 0 0 0;
   0 1 L1/2*cos(q(3)) 0 -1 L2/2*cos(q(6)) 0 0 0 0 0 0 0 0;    %4

   0 0 0 1 0 0.3015*L2*sin(q(6)) -1 0 L3a*sin(q(9))+H3a*cos(q(9)) 0 0 0 0 0 0;
   0 0 0 0 1 -0.3015*L2*cos(q(6)) 0 -1 -L3a*cos(q(9))+H3a*sin(q(9)) 0 0 0 0 0
0;    %6

   0 0 0 0 0 0 1 0 L3c*sin(q(9))-H3a*cos(q(9)) 0 0 0 0 0 0;
   0 0 0 0 0 0 0 1 -L3c*cos(q(9))-H3a*sin(q(9)) 0 0 0 0 0 0;    %8

   0 0 0 0 0 0 1 0 L3b*sin(q(9))+H3b*cos(q(9)) -1 0 L4/2*sin(q(12)) 0 0 0;
   0 0 0 0 0 0 0 1 -L3b*cos(q(9))+H3b*sin(q(9)) 0 -1 -L4/2*cos(q(12)) 0 0 0;
%10

   0 0 0 0 0 0 0 0 0 0 0 1 0 0 -1;
   0 0 0 0 0 0 0 0 0 0 -q(14)-1 q(13)-1 0 q(11)+1 -q(10)+1 0;    %12

   0 0 0 0 0 0 0 0 0 0 0 0 1 0 L5/2*sin(q(15));
   0 0 0 0 0 0 0 0 0 0 0 0 0 1 -L5/2*cos(q(15));    %14

   0 0 1 0 0 0 0 0 0 0 0 0 0 0 0];

```

return

·NEWTON_RAPHSON.m

```

function [q,NR]=Newton_Raphson(q)

n=1000; %max iteraciones
e=0.001; %error permitido

j=1; %contador

NR(:,j)=q;

while (j<n)

    F=restricciones(q);
    J=jacobiana(q);
    q_sig=q-J\F;    %J dividiendo por la izq a F (son matrices)

    if norm(q_sig-q)/norm(q_sig) < e
        q=q_sig;
        NR(:,j+1)=q;
        break;
    else
        q=q_sig;
    end
end

```

```

        NR(:,j+1)=q;
        j=j+1;
    end
end

return

```

·POSICION_TRABAJO.m

```

global L1 L2 L3a H3a L3b H3b L3c L4 L5 H2

L1=4;
L2=4;
L3a=1.6;
L3c=1.4;
H3a=0.3;
L3b=0.192;
H3b=0.589;
L4=1.9161;
L5=1;
H2=0.5;
%anular en restricciones.m -w*t(i)
                                %x3      %y3      %fi3
q=[1.414 1.414 pi/4 3.62 0.992 (293/180)*pi 1.787982 1.021084 pi/4 1.9363 -
0.23666 (39/180)*pi 1.3875 -0.68406 (39/180)*pi]'; %q_aprox

F=restricciones(q) %aprox

[q,NR]=Newton_Raphson(q)

F=restricciones(q) %exacto

```

·DIBUJA.m

```

function []=dibuja(q);

global L1 L2 L3a H3a L3b H3b L3c L4 L5 H2
global Lp Hp

L1=4;
L2=4;
L3a=1.6;
L3c=1.4;
H3a=0.3;
L3b=0.192;
H3b=0.589;
L4=1.9161;
L5=1;
H2=0.5;
Lp=1;
Hp=0.5;

%Bastidor
x1A=0;
y1A=0;

x1B=1;
y1B=0;

```

```

x1C=1;
y1C=-1;

xplot=[x1A x1B x1C];
yplot=[y1A y1B y1C];

plot(xplot,yplot,'*-k', 'LineWidth',3) % *=puntos   -=linea
LineWidth=tamaño

hold on
%Barra1

x1D= q(1)+L1/2*cos(q(3));
y1D= q(2)+L1/2*sin(q(3));

xplot=[0 q(1) x1D];
yplot=[0 q(2) y1D];

plot(xplot,yplot,'*-b', 'LineWidth',3) % *=puntos   -=linea
LineWidth=tamaño

hold on

%Barra2

x1G=q(1)-L1/2*cos(q(3))+q(4)+L2/2*cos(q(6));
y1G=q(2)-L1/2*sin(q(3))+q(5)+L2/2*sin(q(6));

x1H=q(4)+(5/8)*L2*cos(q(6))+H2*sin(q(6));
y1H=q(5)+(5/8)*L2*sin(q(6))-H2*cos(q(6));

xplot=[x1D q(4) x1G x1H];
yplot=[y1D q(5) y1G y1H];

plot(xplot,yplot,'*-r', 'LineWidth',3)

%PLACA_TRIÁNGULO

x1E=q(4)-0.3015*L2*cos(q(6));
y1E=q(5)-0.3015*L2*sin(q(6));

x1F=q(7)-L3b/2*cos(q(9))+H3b*sin(q(9));
y1F=q(8)-L3b/2*sin(q(9))-H3b*cos(q(9));

xplot=[x1B x1E x1F x1B];
yplot=[y1B y1E y1F y1B];

plot(xplot,yplot,'*-g', 'LineWidth',3)

%Eje_Deslizadera

x1J=q(10)-(L4-1)/2*cos(q(12));
y1J=q(11)-(L4-1)/2*sin(q(12));

xJarr=x1J-0.25*sin(q(12));
yJarr=y1J+0.25*cos(q(12));

xJbaj=x1J+0.25*sin(q(12));

```

```
yJbaj=y1J-0.25*cos(q(12));
```

```
xplot=[x1F x1J xJarr xJbaj];
yplot=[y1F y1J yJarr yJbaj];
plot(xplot,yplot,'*-m','LineWidth',3)
```

```
%Caja_Piston
```

```
xplot_rel=[-Lp/2 Lp/2 Lp/2 -Lp/2 -Lp/2];
yplot_rel=[Hp/2 Hp/2 -Hp/2 -Hp/2 Hp/2];
xplot_girado=q(13)+ cos(q(15))*xplot_rel - sin(q(15))*yplot_rel;
yplot_girado=q(14)+ sin(q(15))*xplot_rel + cos(q(15))*yplot_rel;
plot(xplot_girado,yplot_girado,'*-b','LineWidth',3)
```

```
%CDG,apoyos y articulaciones
```

```
circunferencia(q(1),q(2),0.05*Lp,'r'); % CDG L1
circunferencia(q(4),q(5),0.05*Lp,'k'); % CDG L2
circunferencia(q(7),q(8),0.05*Lp,'k'); % CDG L3
circunferencia(q(10),q(11),0.05*Lp,'k'); % CDG L4
circunferencia(q(13),q(14),0.05*Lp,'k'); % CDG L5
```

```
circunferencia(0,0,0.05*Lp,'r'); % Apoyo A
circunferencia(1,0,0.05*Lp,'r'); % Apoyo B
circunferencia(1,-1,0.05*Lp,'r'); % Apoyo C
```

```
circunferencia(x1D,y1D,0.05*Lp,'k'); % articulación D
circunferencia(x1E,y1E,0.05*Lp,'k'); % articulación E
circunferencia(x1F,y1F,0.05*Lp,'k'); % articulación F
```

```
%añadir xlim ylim
```

```
axis square
axis equal
```

```
xlim([-0.5 5.5]);
ylim([-3 5]);
```

```
hold off
return
```

·CIRCUNFERENCIA.m

```
function circunferencia(xc,yc,r,color)
% Función que dibuja una circunferencia con centro en (xc,yc),
% radio r, y color color.

n=20;
delta_theta=2*pi/n;
for i=1:n+1 % Pongo n+1 pq si sólo pongo n no cierra la circunferencia.
    theta=(i-1)*delta_theta;
    x(i)=xc+r*cos(theta);
    y(i)=yc+r*sin(theta);
end
plot(x,y,color);
```

·VELOCIDAD.m

```
function v=velocidad(q)
```



```

% Cálculo del vector velocidad en una posición determinada

global w % Velocidad angular de phi 1

% w=100;

J=jacobiana(q);
VEL=[0;0;0;0;0;0;0;0;0;0;0;0;0;0;w]; % Vector lado derecho ecc velocidad

v=J\VEL;

return

```

·ACELERACIÓN.m

```

function a=aceleracion(q)

% Cálculo de la aceleración en la posición q

global L1 L2 L3a L3c H3a L3b H3b L4 L5

J=jacobiana(q); % Calculamos la jacobiana
v=velocidad(q); % Calculamos el vector de velocidades

% Con q y v definimos el vector gamma de este mecanismo

gamma=[(L1/2*cos(q(3)))*v(3)^2;
        (L1/2*sin(q(3)))*v(3)^2; %2
        -(L1/2*cos(q(3)))*v(3)^2-(L2/2*cos(q(6)))*v(6)^2;
        -(L1/2*sin(q(3)))*v(3)^2-(L2/2*sin(q(6)))*v(6)^2; %4
        (0.3015*L2*cos(q(6)))*v(6)^2+(L3a*cos(q(9))-H3a*sin(q(9)))*v(9)^2;
        (0.3015*L2*sin(q(6)))*v(6)^2+(L3a*sin(q(9))+H3a*cos(q(9)))*v(9)^2;
%6
        (L3c*cos(q(9))+H3a*sin(q(9)))*v(9)^2;
        (L3c*sin(q(9))-H3a*cos(q(9)))*v(9)^2; %8
        (L3b*cos(q(9))-H3b*sin(q(9)))*v(9)^2+(L4/2*cos(q(12)))*v(12)^2;
        (L3b*sin(q(9))+H3b*cos(q(9)))*v(9)^2+(L4/2*sin(q(12)))*v(12)^2;
%10
        0;
        -(v(14))*v(10)^2+(v(13))*v(11)^2+(v(11))*v(13)^2+(-v(10))*v(14)^2;
%12
        (L5/2*cos(q(15)))*v(15)^2;
        (L5/2*sin(q(15)))*v(15)^2; %14
        0];

a=J\-gamma; %%% OJO al signo (-)

return

```

·CINEMÁTICA PALA.m

```

function [pos,v,a]=cinematica_pala

global L1 L2 L3a H3a L3b H3b L3c L4 L5 H2 w t i;

L1=4;
L2=4;
L3a=1.6;
L3c=1.4;

```

```

H3a=0.3;
L3b=0.192;
H3b=0.589;
L4=1.9161;
L5=1;
H2=0.3;

w=100; % Velocidad angular de phi 1

% Vector q aproximado a la posición inicial:

q=[1.414 1.414 pi/4 3.62 0.992 (293/180)*pi 1.787982 1.021084 pi/4 1.9363 -
0.23666 (39/180)*pi 1.3875 -0.68406 (39/180)*pi]'; %q_aprox

giro=pi/8; % Ángulo girado
tfin=giro/w; % Tiempo en dar una vuelta
t = linspace(0,tfin,361); % Crear un vector de tiempos desde 0 hasta el
% tiempo final dividido tantas veces como se desee (por ej divisiones
% de grado).

for i=1:length(t) % Desde que i vale 1 hasta el valor final de t

    [q,NR]=Newton_Raphson(q); % Ejecutamos NR desde la posición anterior
    pos(:,i)=q; % Guardamos la posición nueva

    dibuja(q); % Dibuja la posición
    pause(0.02); % Pausa para visualizar la animación

    v(:,i)=velocidad(q); % Calcula y guarda cada velocidad

    a(:,i)=aceleracion(q); % Calcula y guarda cada aceleración
end

% PLOTEO DE LA POSICIÓN DE CADA VARIABLE EN FUNCIÓN DE t:

figure('name','Cinemática');

subplot(3,3,1);
plot(t,pos(1,:));
xlabel('t');
ylabel('x');
xlim([0 tfin]);
title('Posición x');
hold on

plot(t,pos(4,:));
plot(t,pos(7,:));
plot(t,pos(10,:));
plot(t,pos(13,:));
hold off

subplot(3,3,2);
plot(t,pos(2,:));
xlabel('t');
ylabel('y');
xlim([0 tfin]);
title('Posición y');
hold on

```

```

plot(t,pos(5,:));
plot(t,pos(8,:));
plot(t,pos(11,:));
plot(t,pos(14,:));
hold off

subplot(3,3,3);
plot(t,pos(3,:));
xlabel('t');
ylabel('\phi');
xlim([0 tfin]);
title('Posición \phi');
hold on

plot(t,pos(6,:));
plot(t,pos(9,:));
plot(t,pos(12,:));
plot(t,pos(15,:));
hold off

```

%PLOTEO DE LA VELOCIDAD DE CADA VARIABLE EN FUNCIÓN DE t:

```

subplot(3,3,4)
plot(t,v(1,:));
xlabel('t');
ylabel('x');
xlim([0 tfin]);
title('Velocidad x');
hold on

plot(t,v(4,:));
plot(t,v(7,:));
plot(t,v(10,:));
plot(t,v(13,:));
hold off

subplot(3,3,5)
plot(t,v(2,:));
xlabel('t');
ylabel('y');
xlim([0 tfin]);
title('Velocidad y');
hold on

plot(t,v(5,:));
plot(t,v(8,:));
plot(t,v(11,:));
plot(t,v(14,:));
hold off

subplot(3,3,6)
plot(t,v(3,:));
xlabel('t');
ylabel('\phi');
xlim([0 tfin]);
title('Velocidad \phi');
hold on

plot(t,v(6,:));

```

```

plot(t,v(9,:));
plot(t,v(12,:));
plot(t,v(15,:));
hold off

```

%PLOT DE LA ACELERACIÓN DE CADA VARIABLE EN FUNCIÓN DE t:

% PLOT DE LA ACELERACIÓN DE CADA VARIABLE EN FUNCIÓN DE t:

```

subplot(3,3,7)
plot(t,a(1,:));
xlabel('t');
ylabel('x');
xlim([0 tfin]);
title('Aceleración x');
hold on

```

```

plot(t,a(4,:));
plot(t,a(7,:));
plot(t,a(10,:));
plot(t,a(13,:));
hold off

```

```

subplot(3,3,8)
plot(t,a(2,:));
xlabel('t');
ylabel('y');
xlim([0 tfin]);
title('Aceleración y');
hold on

```

```

plot(t,a(5,:));
plot(t,a(8,:));
plot(t,a(11,:));
plot(t,a(14,:));
hold off

```

```

subplot(3,3,9)
plot(t,a(3,:));
xlabel('t');
ylabel('\phi');
xlim([0 tfin]);
title('Aceleración \phi');
hold on

```

```

plot(t,a(6,:));
plot(t,a(9,:));
plot(t,a(12,:));
plot(t,a(15,:));
hold off

```

return

DERIV_PALA.m

function yp=deriv_pala(t,y) %ESTA FUNCIÓN NO SE EJECUTA

global L1 L2 L3a H3a L3b H3b L3c L4 L5 H2 w t i Lp Hp;

%valores provisionales

```
m1=176.260;  
m2=216.405;  
m3=282.446;  
m4=55.036;  
m5=54.872;
```

```
I1=259.791;  
I2=385.725;  
I3=203.6796;  
I4=27.174;  
I5=26.598;
```

```
q=y(1:15);  
v=y(16:30); %v=q_punto
```

```
M=[m1 0 0 0 0 0 0 0 0 0 0 0 0 0 0;  
    0 m1 0 0 0 0 0 0 0 0 0 0 0 0 0;  
    0 0 I1 0 0 0 0 0 0 0 0 0 0 0;  
    0 0 0 m2 0 0 0 0 0 0 0 0 0 0;  
    0 0 0 0 m2 0 0 0 0 0 0 0 0 0;  
    0 0 0 0 0 I2 0 0 0 0 0 0 0 0 0;  
    0 0 0 0 0 0 m3 0 0 0 0 0 0 0 0;  
    0 0 0 0 0 0 0 m3 0 0 0 0 0 0 0;  
    0 0 0 0 0 0 0 0 I3 0 0 0 0 0 0 0;  
    0 0 0 0 0 0 0 0 0 m4 0 0 0 0 0;  
    0 0 0 0 0 0 0 0 0 0 m4 0 0 0 0;  
    0 0 0 0 0 0 0 0 0 0 0 I4 0 0 0;  
    0 0 0 0 0 0 0 0 0 0 0 0 m5 0 0;  
    0 0 0 0 0 0 0 0 0 0 0 0 0 m5 0;  
    0 0 0 0 0 0 0 0 0 0 0 0 0 0 I5];
```

```
% M=eye(9)*[m1 m1 I1 m2 m2 I2 m3 m3 I3 m4 m4 I4 m5 m5 I5]'; %matriz  
diagonal de arriba
```

%seleccionamos las primeras 14 filas de la matriz jacobiana

```
J=[1 0 L1/2*sin(q(3)) 0 0 0 0 0 0 0 0 0 0 0;  
    0 1 -L1/2*cos(q(3)) 0 0 0 0 0 0 0 0 0 0 0; %2  
  
    1 0 -L1/2*sin(q(3)) -1 0 -L2/2*sin(q(6)) 0 0 0 0 0 0 0 0;  
    0 1 L1/2*cos(q(3)) 0 -1 L2/2*cos(q(6)) 0 0 0 0 0 0 0 0; %4  
  
    0 0 0 1 0 0.3015*L2*sin(q(6)) -1 0 L3a*sin(q(9))+H3a*cos(q(9)) 0 0 0 0 0 0;  
    0 0 0 0 1 -0.3015*L2*cos(q(6)) 0 -1 -L3a*cos(q(9))+H3a*sin(q(9)) 0 0 0 0 0  
0; %6  
  
    0 0 0 0 0 0 1 0 L3c*sin(q(9))-H3a*cos(q(9)) 0 0 0 0 0 0;  
    0 0 0 0 0 0 0 1 -L3c*cos(q(9))-H3a*sin(q(9)) 0 0 0 0 0 0; %8  
  
    0 0 0 0 0 0 1 0 L3b*sin(q(9))+H3b*cos(q(9)) -1 0 L4/2*sin(q(12)) 0 0 0;  
    0 0 0 0 0 0 0 1 -L3b*cos(q(9))+H3b*sin(q(9)) 0 -1 -L4/2*cos(q(12)) 0 0 0;  
%10  
  
    0 0 0 0 0 0 0 0 0 0 1 0 0 -1;  
    0 0 0 0 0 0 0 0 0 -q(14)-1 q(13)-1 0 q(11)+1 -q(10)+1 0; %12  
  
    0 0 0 0 0 0 0 0 0 0 0 1 0 L5/2*sin(q(15));  
    0 0 0 0 0 0 0 0 0 0 0 0 1 -L5/2*cos(q(15))] ; %14
```

```

g=9.8; %gravedad
M1=10;

G=[0; -m1*g; M1;0 ; -m2*g; 0; 0; -m3*g; 0; 0 ; -m4*g; 0; 0; -m5*g; 0];

%seleccionamos las primeras 8 filas de la matriz gamma

gamma=[(L1/2*cos(q(3)))*v(3)^2;
        (L1/2*sin(q(3)))*v(3)^2; %2
        -(L1/2*cos(q(3)))*v(3)^2-(L2/2*cos(q(6)))*v(6)^2;
        -(L1/2*sin(q(3)))*v(3)^2-(L2/2*sin(q(6)))*v(6)^2; %4
        (0.3015*L2*cos(q(6)))*v(6)^2+(L3a*cos(q(9))-H3a*sin(q(9)))*v(9)^2;
        (0.3015*L2*sin(q(6)))*v(6)^2+(L3a*sin(q(9))+H3a*cos(q(9)))*v(9)^2;
%6
        (L3c*cos(q(9))+H3a*sin(q(9)))*v(9)^2;
        (L3c*sin(q(9))-H3a*cos(q(9)))*v(9)^2; %8
        (L3b*cos(q(9))-H3b*sin(q(9)))*v(9)^2+(L4/2*cos(q(12)))*v(12)^2;
        (L3b*sin(q(9))+H3b*cos(q(9)))*v(9)^2+(L4/2*sin(q(12)))*v(12)^2;
%10
        0;
        -(v(14))*v(10)^2+(v(13))*v(11)^2+(v(11))*v(13)^2+(-v(10))*v(14)^2;
%12
        (L5/2*cos(q(15)))*v(15)^2;
        (L5/2*sin(q(15)))*v(15)^2]; %14

%seleccionamos las primeras 8 filas de la matriz F
F=[q(1)-L1/2*cos(q(3));
    q(2)-L1/2*sin(q(3)); %2

    q(1)+L1/2*cos(q(3))-q(4)+L2/2*cos(q(6));
    q(2)+L1/2*sin(q(3))-q(5)+L2/2*sin(q(6)); %4

    q(4)-0.3015*L2*cos(q(6))-q(7)-L3a*cos(q(9))+H3a*sin(q(9));
    q(5)-0.3015*L2*sin(q(6))-q(8)-L3a*sin(q(9))-H3a*cos(q(9)); %6

    q(7)-L3c*cos(q(9))-H3a*sin(q(9))-1;
    q(8)-L3c*sin(q(9))+H3a*cos(q(9)); %8

    q(7)-L3b/2*cos(q(9))+H3b*sin(q(9))-q(10)-L4/2*cos(q(12));
    q(8)-L3b/2*sin(q(9))-H3b*cos(q(9))-q(11)-L4/2*sin(q(12)); %10

    q(12)-q(15);
    q(13)*q(11)-q(13)*q(14)-q(11)+q(14)-q(10)*q(14)-q(10)+q(13)*q(14)+q(13);
%12

    q(13)-L5/2*cos(q(15))-1;
    q(14)-L5/2*sin(q(15))+1]; %14

wn=10;
chi=2;

gamma_est=gamma + 2*chi*wn*J*v + wn^2*F; %gamma estabilizado

A=[M -J';
    J zeros(14)] ; %zeros(14)= matriz de 14x14 0

b=[G; -gamma_est];

```

```
sol=A\b;
```

```
qpp=sol(1:15);  
yp=[v;qpp];
```

```
return
```

·DINÁMICA PALA.m

```
function [tout,yout]=dinamica_pala %t_out y_out
```

```
global L1 L2 L3a H3a L3b H3b L3c L4 L5 H2 Lp Hp; % w t i no forman parte en  
dinámica
```

```
L1=4;  
L2=4;  
L3a=1.6;  
L3c=1.4;  
H3a=0.3;  
L3b=0.3008;  
H3b=0.5857;  
L4=1.9161;  
L5=1;  
H2=0.3;  
Lp=1;  
Hp=0.5;
```

```
tfin=0.5; %tiempo fin simulacion  
dt=0.005; %paso del tiempo  
t=0:dt:tfin;
```

```
q=[1.4142 1.4142 0.7854 3.6208 0.9921 5.1198 2.2100 0.7654 0.7751 1.7733 -  
0.3678 0.6896 1.3857 -0.6819 0.6896]'; %posicion (15 elementos) (q_0)  
qp=[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]'; %velocidad (15 elementos) (q_punto_0)
```

```
y=[q;qp];
```

```
[tout,yout]=ode23('deriv_pala',t,y); %ode23 mas rapido que ode 45 pero  
menos preciso
```

```
for i=1:length(tout)  
    y=yout(i,:);  
    q=y(1:15);  
  
    dibuja(q)  
    pause(dt);  
    %Aceleración  
    t=tout(i);  
    yp=deriv_pala(t,y);  
    qpp=yp(16:30);  
    a(:,i)=qpp;  
end  
yout=[yout,a'];
```

```
% PLOTEO DE LA POSICIÓN DE CADA VARIABLE EN FUNCIÓN DE t:
```

```
figure('name','Dinámica');
```

```

subplot(3,3,1);
plot(tout,yout(:,1));
xlabel('t');
ylabel('x');
xlim([0 tfin]);
title('Posición x');
hold on

plot(tout,yout(:,4));
plot(tout,yout(:,7));
plot(tout,yout(:,10));
plot(tout,yout(:,13));
hold off

subplot(3,3,2);
plot(tout,yout(:,2));
xlabel('t');
ylabel('y');
xlim([0 tfin]);
title('Posición y');
hold on

plot(tout,yout(:,5));
plot(tout,yout(:,8));
plot(tout,yout(:,11));
plot(tout,yout(:,14));
hold off

subplot(3,3,3);
plot(tout,yout(:,3));
xlabel('t');
ylabel('\phi');
xlim([0 tfin]);
title('Posición \phi');
hold on

plot(tout,yout(:,6));
plot(tout,yout(:,9));
plot(tout,yout(:,12));
plot(tout,yout(:,15));
hold off

%VELOCIDADES

subplot(3,3,4);
plot(tout,yout(:,16));
xlabel('t');
ylabel('x');
xlim([0 tfin]);
title('Velocidad x');
hold on

plot(tout,yout(:,19));
plot(tout,yout(:,22));
plot(tout,yout(:,25));
plot(tout,yout(:,28));
hold off

subplot(3,3,5);

```



```

plot(tout,yout(:,17));
xlabel('t');
ylabel('y');
xlim([0 tfin]);
title('Velocidad y');
hold on

plot(tout,yout(:,20));
plot(tout,yout(:,23));
plot(tout,yout(:,26));
plot(tout,yout(:,29));

subplot(3,3,6);
plot(tout,yout(:,18));
xlabel('t');
ylabel('\phi');
xlim([0 tfin]);
title('Velocidad \phi');
hold on

plot(tout,yout(:,21));
plot(tout,yout(:,24));
plot(tout,yout(:,27));
plot(tout,yout(:,30));
hold off

%ACELERACIONES

subplot(3,3,7);
plot(tout,yout(:,31));
xlabel('t');
ylabel('x');
xlim([0 tfin]);
title('Aceleración x');
hold on

plot(tout,yout(:,34));
plot(tout,yout(:,37));
plot(tout,yout(:,40));
plot(tout,yout(:,43));
hold off

subplot(3,3,8);
plot(tout,yout(:,32));
xlabel('t');
ylabel('y');
xlim([0 tfin]);
title('Aceleración y');
hold on

plot(tout,yout(:,35));
plot(tout,yout(:,38));
plot(tout,yout(:,41));
plot(tout,yout(:,44));

subplot(3,3,9);
plot(tout,yout(:,33));
xlabel('t');
ylabel('\phi');

```

```
xlim([0 tfin]);  
title('Aceleración \phi');  
hold on  
  
plot(tout,yout(:,36));  
plot(tout,yout(:,39));  
plot(tout,yout(:,42));  
plot(tout,yout(:,45));  
hold off  
  
end
```